

DIAGNOSTIC REPORT — UPDATED

Ruhvi Performance Audit

Foundational architecture & core codebase analysis — latency bottlenecks and optimization opportunities for the Ruhvi fine-jewellery e-commerce platform.

Project: Ruhvi E-Commerce (Next.js 15, App Router)

Scope: Implemented modules only — frontend, assets, third-party dependencies, API/data layer, infrastructure

Initial Audit: 30 August 2026 • **This Revision:** 30 August 2026 (reflects current codebase)

Table of Contents

1. Executive Summary
2. Codebase Changes Since Initial Audit
3. 1. Frontend Code

4. 2. Asset Management
5. 3. Third-Party Dependencies
6. 4. API & Data Fetching
7. 5. Infrastructure & Delivery
8. Bundle Size Analysis
9. Strategic Optimization Roadmap

Executive Summary

Ruhvi is a Next.js 15 (App Router) e-commerce application. The audit identified **13 High**, **12 Medium**, and **5 Low** severity issues across the five audited areas. The most consequential problems are architectural rather than cosmetic:

- The **root layout is forced dynamic** by an unnecessary `headers()` call, disabling static generation / ISR for every page.
- The **middleware performs authentication + RBAC database lookups on every public request**, inflating TTFB app-wide.
- Five analytics tools and three push-notification/auth systems load on every page, directly taxing Time to Interactive (TTI) and Total Blocking Time (TBT).
- Core catalog and auth flows make **heavy, redundant client-side Supabase queries** with no caching layer.
- No CDN caching headers or ISR configuration means almost nothing is served from the edge.

All of these are addressable during the development phase — the cost of fixing them now is a fraction of the cost after production traffic arrives.

Codebase Changes Since Initial Audit

The following changes were made to the repository after the initial audit pass. They do not alter the core audit findings above but are reflected throughout this revision where relevant:

- **Test infrastructure added:** `jest` + `ts-jest` dev dependencies, `test` / `test:watch` npm scripts, and `jest.config.js` with path aliasing. An AI routing test suite (`src/lib/ai/__tests__/routing.test.ts`) now mocks `next/headers` and `@supabase/ssr` for unit testing.
- **Middleware fail-closed behavior (H-5.1 related):** `src/middleware.ts` now redirects to login when session/RBAC verification fails on internal routes instead of passing the request through. The broader performance concern — RBAC database lookups running on every public request — remains unchanged.
- **AI error classifier refinement (internal):** `src/lib/ai/error-classifier.ts` now classifies 5xx server errors before rate-limit patterns, and maps Gemini `RESOURCE_EXHAUSTED` to `QUOTA_EXHAUSTED` instead

of `RATE_LIMIT`, improving credential-rotation decisions. No frontend performance impact.

- **Order fail-closed behavior (internal):** `src/lib/orders/create-order.ts` now throws `OrderError` if wallet debit fails during checkout instead of silently continuing — a correctness fix with no frontend impact.

1. Frontend Code

HIGH H-1.1 — Root Layout Forced Dynamic via `headers()`

`src/app/layout.tsx:147` calls `await headers()` to determine subdomain routing. This marks the **entire root layout** as dynamic, defeating Next.js static optimization (SSG/ISR) for **every page**. Every navigation triggers a full server round-trip, so no page is ever edge-cached.

Fix: Move subdomain detection to middleware (already implemented there) and remove it from the layout. Pass the detected host via an `x-host` header or `cookies()` instead.

HIGH H-1.2 — Global `'use client'` Footer

`src/components/layout/Footer.tsx:1` is marked `'use client'` despite having no interactivity beyond the newsletter form. This forces the entire footer into the client JS bundle on every page, contributing to the 189 kB baseline shared chunk.

Fix: Split the newsletter form into a small separate `'use client'` island and keep the rest of the footer as a server component.

HIGH H-1.3 — CustomerSupportChat Mounted Globally

`src/components/CustomerSupportChat.tsx` (629 lines — full chat widget with drag-and-drop, resize, AI integration, and typewriter effects) is rendered unconditionally on every public page via `src/app/layout.tsx:255`. It adds ~30 kB of JS to the critical path, blocks hydration, and its `BotMessage` typewriter effect (10 ms interval per message) creates avoidable main-thread work.

Fix: Lazy-load via `next/dynamic` with `ssr: false`; mount only after user interaction (clicking the mascot).

HIGH H-1.4 — Five Google Fonts Loaded Globally

`src/app/layout.tsx:3-8` loads Inter, Playfair_Display, Jost, Marcellus, and Cormorant_Garamond (5 families, some with multiple weights and italic variants). Although `next/font` self-hosts them, the cumulative download is ~120 kB+ on every page. Only three are actually used in CSS (Jost for body, Cormorant for headings, Marcellus for `.display`).

Fix: Remove unused fonts; keep Jost (body), Cormorant Garamond (headings), Marcellus (display). Set `display: 'swap'`.

MEDIUM M-1.1 — 28 kB `globals.css` with `!important` Dark Mode Overrides

`src/app/globals.css` (1407 lines) contains ~250 lines of `html.dark .class { ... !important }` overrides (~80% of the dark-mode section). This indicates the CSS architecture is fighting itself — Tailwind's `dark:` variant is not used consistently, and the `!important` cascade prevents proper tree-shaking and makes styling brittle.

Fix: Migrate dark mode to Tailwind's built-in `dark:` prefix throughout; remove the `html.dark` blocks from `globals.css`.

MEDIUM M-1.2 — Repeated Inline SVGs on Homepage

`src/app/page.tsx` inlines the same arrow SVG (`<path d="M5 12h14...">`) 7 times across the hero, promo cards, category cards, and CTAs, adding redundant markup per occurrence.

Fix: Extract into a shared server component or use `lucide-react` consistently.

MEDIUM M-1.3 — SpatialPage Dead Props

`src/components/design-system/SpatialPage.tsx` accepts `showParticles` and `showOrbs` props but never uses them. `ProductsCatalogClient.tsx` passes these, creating the expectation of particle/orb effects that never render.

Fix: Implement the effects or remove the props.

LOW L-1.1 — Unused Global `@keyframes`

Mascot animations (`mascot-bob` , `mascot-wiggle` , `mascot-bounce` , `mascot-glow-pulse` , etc.) are defined globally in `globals.css` (~100 lines) but only used by the lazily-loaded BotMascot inside CustomerSupportChat — always downloaded.

Fix: Move mascot-specific animations into a CSS module co-located with the component.

2. Asset Management

HIGH H-2.1 — Large Unoptimized Local Images

- `public/images/admin-login.png` — 286 kB
- `public/images/logo.png` — 253 kB
- `public/images/categories/bridal.jpg` — 128 kB
- `public/images/categories/pendants.jpg` — 135 kB

These are served as raw files without compression, WebP conversion, or responsive sizing, and they bypass the `next/image` pipeline entirely.

Fix: Optimize via `sharp` at build time, convert to WebP/AVIF, and route through `next/image` with proper `sizes`.

HIGH H-2.2 — No Width Constraints on Cloudinary URLs

Homepage and related imagery (`src/app/page.tsx`) use Cloudinary URLs with `f_auto,q_auto` but **no explicit width transformation**. The browser downloads full-resolution originals (~2000+ px wide) and downscales via CSS, wasting bandwidth and decode time.

Fix: Inject width transformations (`w_600` , `w_1200` , `w_1920`) matched to the `sizes` attribute on every Cloudinary URL.

MEDIUM M-2.1 — Logo Uses `priority` on Every Page

`src/components/layout/Navbar.tsx:135-141` sets `priority` on the logo image. Priority is only needed on the initial page load; on client-side navigations it forces an avoidable preload.

Fix: Use `loading="eager"` for the logo, or set priority only on first page load.

MEDIUM M-2.2 — No Format Negotiation for Local/Unsplash Images

Cloudinary URLs benefit from `f_auto` format negotiation, but local images and Unsplash sources have no automatic WebP/AVIF conversion.

Fix: Route all imagery through Cloudinary fetch or enable image format optimization in `next.config.js`.

LOW L-2.1 — Missing Favicon Preload

No `<link rel="preload" as="image">` for `/favicon.ico`. Negligible impact.

Fix: Optional; add preload if the favicon is above the fold.

3. Third-Party Dependencies

HIGH H-3.1 — Five Analytics/Marketing Scripts on Every Page

`src/app/layout.tsx` mounts five tracking tools simultaneously on every public page:

- Google Analytics (`gtag.js`)
- Meta Pixel (`fbevents.js`)
- Microsoft Clarity
- PostHog (with session recording enabled)
- Vercel SpeedInsights

Combined, these add ~80–120 kB of uncompressed JS, 5+ HTTP connections, and compete for the main thread during hydration — directly increasing TBT and delaying TTI.

Fix: Consolidate to a single analytics platform (PostHog can replace GA + Clarity). Load Meta Pixel only on commerce pages. Reduce Clarity/PostHog session-recording sampling in production.

HIGH H-3.2 — Three Push/Auth Systems (Firebase Auth + FCM + OneSignal)

The layout initializes Firebase Auth (via `AuthContext`), Firebase Cloud Messaging (`FcmInit`), and OneSignal (`OneSignalInit`). Firebase pulls in the full `firebase/auth` module (~50 kB gzipped) on every page load; OneSignal adds its own ~30 kB SDK. That is three overlapping systems for one push-notification need.

Fix: Choose one provider. Firebase Auth + FCM alone can cover authentication and push; remove the OneSignal SDK entirely, or drop Firebase in favor of Supabase Auth + a single push provider.

HIGH H-3.3 — `react-three/fiber` + `three` + `drei` in Bundle

`package.json` includes `@react-three/fiber` , `@react-three/drei` , and `three` (~500 kB+ gzipped). Build output shows `/account` at 361 kB and `/admin/ai-settings` at 345 kB First Load JS — strong evidence these libraries leak into client bundles.

Fix: Audit imports to guarantee Three.js loads only via `next/dynamic` with `ssr: false` on specific product pages, never in the shared layout chunk.

MEDIUM M-3.1 — PhotoSwipe CSS Imported in Client Component

`src/components/products/ProductImageGallery.tsx:7` imports `photoswipe/dist/photoswipe.css` , pulling ~15 kB of CSS into the shared bundle for a lightbox used only on product-detail pages.

Fix: Dynamic-import the component and inject its CSS only when mounted.

MEDIUM M-3.2 — `recharts` Charting Library

`recharts` (~200 kB) is a dependency. Confirm it is limited to admin/report pages and not accidentally included in the main customer bundle.

Fix: Verify chunk attribution via a bundle analyzer; scope it to admin routes.

LOW L-3.1 — Icon Imports in Footer

`lucide-react` is tree-shakeable via named imports (correctly used), but the Footer imports 7 icons. Review whether all are needed above the fold.

Fix: Trim unused icons; keep only those rendered.

4. API & Data Fetching

HIGH H-4.1 — Catalog Fetches Everything Client-Side

`src/app/products/ProductsCatalogClient.tsx:52-69` runs a client-side Supabase query with `select('*', images:product_images(*), category:categories('*'))` — a heavy relational join on every request, plus `count: 'exact'` (a full table scan) on every pagination page. The catalog page is **entirely client-rendered**: users see a spinner until JS downloads, hydrates, and the query completes. Filtering/sorting is reimplemented in the browser instead of pushed to the database.

Fix: Server-side fetch with React Suspense streaming; use `useInfiniteQuery` with caching; avoid `count: 'exact'`; push filters/sort/limit into the query.

HIGH H-4.2 — AuthContext Runs up to 4 Sequential Queries

`src/context/AuthContext.tsx:54-110` may fire four sequential `users` lookups (by ID, by Firebase UID via RPC, by phone, by email) before falling back to a default profile. On every page load for authenticated users this adds 4+ Supabase round trips, worsening TTFB and TTI.

Fix: Consolidate into a single indexed query/RPC that accepts all identifiers and returns the first match. Add a composite index on `(firebase_uid, id, email, phone)`.

HIGH H-4.3 — Search Queries Supabase on Every Keystroke

`src/components/search/SearchBar.tsx:75-108` fires an `ilike` query on every keystroke (300 ms debounce). `ilike` is not index-friendly and scans the full `products` table; the query also joins `product_images` and `categories` each time.

Fix: Use Postgres full-text search (`to_tsvector`) with a GIN index, or an external search service (Meilisearch/Typesense). Add a composite GIN index on `(name, sku)`.

HIGH H-4.4 — No Caching on Homepage Data

`src/app/page.tsx:64-79` makes three Supabase queries (categories, collections, settings) on every request with no caching, even though this data is effectively static between admin edits.

Fix: Use `unstable_cache` / `revalidate` for homepage data; set page-level `revalidate` for ISR.

MEDIUM M-4.1 — Redundant Product-Detail Queries

`src/app/products/[slug]/page.tsx:88-103` queries by slug, then by ID on failure — in both the page and `generateMetadata`. The same dataset is fetched twice per request, and related products trigger up to two more full joins.

Fix: Share one cached fetch between metadata and page; add a DB index on `slug`.

MEDIUM M-4.2 — Full Product Objects in `localStorage`

`src/context/CartContext.tsx:43-47` serializes entire `Product` objects (including images, category, description) into `ruhvi_cart_v1`, bloating `localStorage` and slowing every `JSON.parse` / `JSON.stringify`.

Fix: Persist only `product_id`, `quantity`, `price_at_add`; hydrate full product data from a cache/SWR store.

LOW L-4.1 — `DEMO_PRODUCTS` Statically Imported in Several Components

The 8.5 kB demo dataset is imported in `ProductsCatalogClient.tsx`, `SearchBar.tsx`, and product-detail files, adding weight even when live Supabase data is available.

Fix: Dynamic-import or remove the demo fallback once production data is stable.

5. Infrastructure & Delivery

HIGH H-5.1 — Middleware Blocks Every Route with Auth + RBAC

`src/middleware.ts` matches all non-static paths. For every public page view it: sets security headers, performs subdomain routing, creates a Supabase server client with the service-role key, verifies the session cookie, and fetches `role`, `account_status`, `allowed_portals` from `users` — **2–3 database round trips added to every request, including the public homepage**. This directly raises TTFB by ~100–300 ms app-wide.

Change since initial audit: The middleware now fails closed — if session/RBAC verification throws on an internal route, it redirects to login instead of passing the request through. This is a correctness improvement, but the performance impact of running RBAC database lookups on every public request is unchanged.

Fix: Narrow the matcher to internal routes only (`/admin` , `/support` , `/operations` , `/marketing` , `/portal-orders`). Do not run auth/RBAC middleware on public pages; move security headers to `next.config.js` .

HIGH H-5.2 — No Caching Headers on Any Response

The middleware sets security headers but no `Cache-Control` , `CDN-Cache-Control` , or `Surrogate-Control` . `vercel.json` contains only a cron job — no CDN rules. Nothing is cached at the edge.

Fix: Add `Cache-Control: public, max-age=3600, s-maxage=86400, stale-while-revalidate=604800` to public pages; configure Vercel CDN rules.

HIGH H-5.3 — No ISR Configuration

No `revalidate` or `generateStaticParams` anywhere; the dynamic root layout (H-1.1) forces per-request server rendering for every route, so there is no edge caching and higher origin load.

Fix: Set `export const dynamic = 'force-static'` where safe; use `revalidate` on homepage and product pages; remove the `headers()` call from the root layout.

MEDIUM M-5.1 — Hardcoded Supabase Credentials in Source

`src/lib/supabase/server.ts:7-8` and `src/lib/supabase/client.ts:13-17` embed fallback `NEXT_PUBLIC_SUPABASE_URL` / `ANON_KEY` strings, meaning staging/production share the same database if env vars are missing.

Fix: Remove hardcoded fallbacks; fail fast when environment variables are absent.

MEDIUM M-5.2 — Unnecessary `outputFileTracingRoot`

`next.config.js:3` sets `outputFileTracingRoot: __dirname`, which is unnecessary for Vercel and can cause incorrect trace output.

Fix: Remove unless using a custom Docker deployment.

LOW L-5.1 — `vercel.json` Has Only a Cron Job

No headers, redirects, or cache rules in `vercel.json`. Default CDN behavior caches nothing for dynamic routes.

Fix: Add `headers` and cache rules for static assets and public pages.

Bundle Size Analysis (from build.log)

Route	First Load JS	Notes
/ (home)	206 kB	High for a mostly-static page
/account	361 kB	Very high — admin/dashboard deps leaking in
/login	305 kB	Extremely high for a login form
/checkout	314 kB	High — includes checkout logic
/products	211 kB	High for a catalog page
/products/[slug]	296 kB	High — PhotoSwipe, 3D viewer, etc.
/admin/ai-settings	345 kB	Very high
/contact	223 kB	High for a contact form
Baseline shared chunk	189 kB	Loaded on every page

The 189 kB baseline includes all five fonts, the client Navbar/Footer, all five context providers, five analytics tools, three push/auth systems, the support chat widget, toast/offline providers, and SpeedInsights. Slashing the shared chunk is the single highest-leverage win for TTI/TBT.

Note: build.log was removed from the repository during the scratch-file cleanup; the table above reflects the last recorded build output and should be re-validated with a fresh `next build`.

Strategic Optimization Roadmap

Phase 1 — High Impact / Low Effort (Week 1)

#	Change	Expected Impact
P1	Remove <code>headers()</code> from root layout	–50–150 ms TTFB; re-enables SSG/ISR
P2	Narrow middleware matcher to internal routes	Removes 2–3 DB round trips per public view
P3	Lazy-load CustomerSupportChat (<code>next/dynamic</code>)	Shared chunk 189 kB → ~150 kB
P4	Remove unused fonts (keep 2–3)	–80 kB font download per page
P5	Add <code>Cache-Control</code> headers on public pages	Edge caching; origin load –90%+

Phase 2 — High Impact / Medium Effort (Week 2)

#	Change	Expected Impact
P6	Consolidate analytics → PostHog only (drop GA + Clarity)	–3+ HTTP requests, ~50 kB JS
P7	Remove OneSignal; keep Firebase FCM only	–30 kB JS
P8	Footer → server component + client newsletter island	–4 kB shared JS
P9	ISR on homepage, catalog, product detail	Edge caching for ~95% of views
P10	Migrate dark mode to Tailwind <code>dark:</code> prefix	Removes ~250 lines of <code>!important</code>

Phase 3 — Medium Impact / Medium Effort (Week 3)

#	Change	Expected Impact
P11	Server-side catalog with streaming	–1–3 s TTI on /products
P12	Full-text search index or external search	Search latency 300 ms+ → <50 ms
P13	Consolidate auth profile resolution to one indexed query	Auth 4 round trips → 1
P14	Optimize/compress local images (WebP)	–60–80% image payload
P15	Width transformations on all Cloudinary URLs	–50–70% image download size

Phase 4 — Ongoing / Maintenance

#	Change	Expected Impact
P16	Add <code>@next/bundle-analyzer</code> to track regressions	Prevents bundle bloat
P17	Core Web Vitals monitoring	Data-driven optimization
P18	Lighthouse CI in PRs	Maintains performance baseline
P19	Prune unused npm dependencies	Smaller installs, smaller attack surface
P20	Consolidate three push/auth systems to one	Less maintenance, less JS